

Indian Startup Funding

Machine Learning — Step-by-Step Modelling Guide

Feature Engineering · Three Algorithms · Best Model · SHAP Interpretation

Dataset	startup_funding_ml_ready.csv · 2,879 rows · 84 columns
ML Tasks	① Regression — predict log(Funding Amount) ② Classification — predict Funding Stage
Algorithms	Ridge Regression (baseline) · Random Forest · XGBoost
Explains	SHAP (SHapley Additive exPlanations) for best model interpretation
Target	Young entrepreneurs asking: how much can I raise? At what stage?

■ CRITICAL DATA ISSUES FOUND IN THIS FILE — FIXED IN PART 1

①	50 DUPLICATE encoded columns (.1/.2/.3 suffixes) — all must be dropped before any model
②	log_Amount is CORRUPTED — NaN funding rows silently filled with 0.0 (log1p of NaN=0)
③	3 EXTREME outliers in Amount_M (> \$10B) — require Winsorization (capping)
④	Stage column has 70+ inconsistent labels — 'Seed Round', 'seed funding' all mean 'Seed'
⑤	Sector encoded columns still contain pre-cleaning duplicates (sec_EdTech vs sec_Edtech)

Table of Contents

PART 1 – Data Audit & Fixing Inherited Issues

- 1.1 Load the File & Run Instant Audit
- 1.2 Drop 50 Duplicate Encoded Columns
- 1.3 Fix the Corrupted log_Amount Column
- 1.4 Winsorise Extreme Outliers in Amount_M
- 1.5 Consolidate Messy Stage Labels

PART 2 – Feature Engineering

- 2.1 Re-Encode Sector & City Cleanly
- 2.2 Ordinal Encode the Consolidated Stage
- 2.3 Engineer New Features (Age, Tier-1, Decade)
- 2.4 Final Feature Matrix & Target Variables
- 2.5 Visualise Feature Correlations

PART 3 – Model Building — Three Algorithms

- 3.1 Setup: Imports, Pipelines & Metrics
- 3.2 Task A — Regression: Predict Funding Amount
 - Model A1 · Ridge Regression (Baseline)
 - Model A2 · Random Forest Regressor
 - Model A3 · XGBoost Regressor
- 3.3 Task B — Classification: Predict Funding Stage
 - Model B1 · Logistic Regression (Baseline)
 - Model B2 · Random Forest Classifier
 - Model B3 · XGBoost Classifier
- 3.4 Cross-Validation & Model Comparison Dashboard

PART 4 – Predictions with the Best Model

- 4.1 Select the Winner (Regression & Classification)
- 4.2 Hyperparameter Tuning with GridSearchCV
- 4.3 Final Evaluation on Held-Out Test Set
- 4.4 Residual Analysis (Regression)
- 4.5 Confusion Matrix & Classification Report
- 4.6 Make New Predictions on Custom Input

PART 5 – Best Model Interpretation — SHAP

- 5.1 What is SHAP and Why It Matters
- 5.2 Compute SHAP Values
- 5.3 Global Importance — SHAP Summary Plot
- 5.4 SHAP Bar Plot (Mean Absolute Impact)
- 5.5 SHAP Dependence Plots (Feature Interactions)
- 5.6 Local Explanation — Waterfall Plot for One Prediction
- 5.7 Force Plot — Visualising a Single Prediction's Drivers

PART 6 – Save & Deploy

- 6.1 Save the Trained Model (joblib + pickle)
- 6.2 Load Model & Predict on New Data
- 6.3 Export All Results to Drive

PART 1

Data Audit & Fixing Inherited Issues

■ Before building any model, every ML project must start with a data audit. The file you have (startup_funding_ml_ready.csv) was saved after the EDA phase and contains five critical issues that will silently ruin any model you build without fixing. This part addresses all of them.

1.1 Load the File & Run an Instant Audit

The first cell of your ML notebook should always be a quick audit — shape, dtypes, nulls, and a scan for obvious problems.

```
# CELL 1 – Install & imports

!pip install xgboost shap --quiet

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

from IPython.display import display, HTML

warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-whitegrid')
pd.set_option('display.max_columns', 30)
pd.set_option('display.float_format', '{:,.4f}'.format)
```

```
# CELL 2 – Load & audit

df = pd.read_csv('startup_funding_ml_ready.csv')

display(HTML('<h3>Dataset Shape</h3>'))
display(HTML(f'<b>Rows:</b> {df.shape[0]:,} | <b>Cols:</b> {df.shape[1]}'))

# ■■ Audit 1: Duplicate encoded columns (.1, .2, .3 suffixes) ■■
dup_cols = [c for c in df.columns
             if any(f'.{i}' in c for i in [1,2,3])]
display(HTML(f'<b style="color:red">Duplicate encoded columns found: {len(dup_cols)}</b>'))
display(dup_cols[:8]) # show first 8 as sample

# ■■ Audit 2: Corrupted log_Amount ■■
fake_zeros = df[df['Amount_M'].isna() & (df['log_Amount'] == 0)]
display(HTML(f'<b style="color:red">Corrupted log_Amount rows (NaN→0): {len(fake_zeros)}</b>'))

# ■■ Audit 3: Extreme outliers ■■
display(HTML(f'<b>Amount_M > $10B:</b> {(df["Amount_M"] > 10000).sum()} rows'))
```

```
# ■■ Audit 4: Stage label chaos ■■

display(HTML(f'<b>Unique Stage labels:</b> {df["Stage"].nunique()}'))

display(df['Stage'].value_counts().head(20).to_frame('Count'))

# ■■ Audit 5: Null summary ■■

null_pct = (df.isnull().sum() / len(df) * 100).round(1)

display(HTML('<h4>Columns with > 5% nulls</h4>'))

display(null_pct[null_pct > 5].sort_values(ascending=False).to_frame('Null %'))
```

Reason: Running an audit before touching the data prevents building a model on bad foundations. Each audit check is targeted — duplicate columns inflate dimensionality and cause multicollinearity, corrupted zeros skew the target distribution, extreme outliers dominate loss functions in regression, and inconsistent stage labels mean your classifier is trying to distinguish 'Seed' from 'Seed Round' from 'seed funding' — which are all the same thing.

1.2 Drop the 50 Duplicate Encoded Columns

■ The 50 columns with .1, .2, .3 suffixes are artefacts of the `pd.get_dummies()` being applied to each year's DataFrame separately before merging. They are exact duplicates and **MUST** be removed — keeping them gives your model 50 redundant signals that cause data leakage and inflate training time.

```
# CELL 3 – Drop duplicate encoded columns

dup_cols = [c for c in df.columns
if any(f'.{i}' in c for i in [1,2,3])]

df.drop(columns=dup_cols, inplace=True)

display(HTML(f'Shape after dropping duplicates: <b>{df.shape}</b>'))

display(HTML('<h4>Remaining Columns</h4>'))

display(df.columns.tolist())
```

Reason: `df.drop(inplace=True)` modifies the DataFrame in-place so we don't create an extra copy in memory. We use the same list comprehension from the audit to guarantee we drop exactly what we identified. After this step the dataset shrinks from 84 to 34 columns.

1.3 Fix the Corrupted log_Amount Column

When `log_Amount` was created in the EDA phase, NaN `Amount_M` values were silently treated as 0 by `log1p`. This means 706 rows have a fake `log_Amount=0.0` even though their true funding amount is unknown. We must recompute it correctly.

```
# CELL 4 – Recompute log_Amount correctly

# np.log1p propagates NaN naturally – NaN input → NaN output
# This correctly marks unknown amounts as NaN instead of 0

df['log_Amount'] = np.log1p(df['Amount_M']) # NaN stays NaN

# Verify the fix

before_fix = 706 # known from audit

after_fix = df['log_Amount'].isna().sum()
```

```
display(HTML(f'''
<b>Before fix:</b> 706 corrupted zeros in log_Amount<br>
<b>After fix:</b> {after_fix} proper NaN values<br>
<b>Valid log_Amount rows:</b> {df["log_Amount"].notna().sum():,}
'''))

display(HTML('<h4>log_Amount Distribution (valid rows only)</h4>'))

display(df['log_Amount'].dropna().describe().to_frame())
```

Reason: The regression target variable MUST be accurate. A model trained with 706 fake zeros at the bottom of the target distribution will learn a biased decision boundary — it will systematically under-predict funding for startups that resemble the ones with missing amounts. $\text{np.log1p}(\text{NaN}) = \text{NaN}$ by IEEE floating-point rules, which is the correct behaviour here.

1.4 Winsorise Extreme Outliers in Amount_M

Three rows have Amount_M > \$10 billion — likely data-entry errors or very unusual mega-rounds. These single rows would dominate the regression loss function and warp all coefficient estimates.

```
# CELL 5 – Winsorise at the 99.5th percentile
# Winsorisation CAPS outliers instead of removing rows,
# so we don't lose any observations.

cap = df['Amount_M'].quantile(0.995) # ~$1.15B for this dataset
display(HTML(f'Winsorisation cap (99.5th pct): <b>${cap:,.1f}M</b>'))

df['Amount_M'] = df['Amount_M'].clip(upper=cap)
df['log_Amount'] = np.log1p(df['Amount_M']) # recompute after capping

# Visualise before vs after
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

valid = df['log_Amount'].dropna()
axes[0].hist(valid, bins=50, color='#3B82F6', edgecolor='white', alpha=0.85)
axes[0].set_title('log(Amount) After Winsorisation', fontsize=12, fontweight='bold')
axes[0].set_xlabel('log(1 + Amount_M)')
axes[0].set_ylabel('Frequency')

axes[1].boxplot(valid, vert=True, patch_artist=True,
boxprops=dict(facecolor='#1B7F79', color='navy'))
axes[1].set_title('Box Plot – log(Amount) After Winsorisation',
fontsize=12, fontweight='bold')
axes[1].set_ylabel('log(1 + Amount_M)')

plt.suptitle('Target Variable Distribution After Cleaning',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('target_distribution.png', dpi=150, bbox_inches='tight')
```



```
df['Stage_clean'] = df['Stage'].map(stage_map)

# Rows that didn't map get NaN — show what fell through
unmapped = df[df['Stage_clean'].isna() & df['Stage'].notna()]['Stage'].unique()
display(HTML(f'Unmapped stage values: <b>{len(unmapped)}</b>'))
display(unmapped[:10]) # inspect any remaining

display(HTML('<h4>Consolidated Stage Distribution</h4>'))
display(df['Stage_clean'].value_counts().to_frame('Count'))
```

Reason: We use .map() instead of .replace() because map returns NaN for any key not in the dictionary — this makes unmapped values visible rather than silently keeping the raw label. We also group Series C through I into 'Series C+' because each individual late stage has very few samples (< 30), which is not enough for a classifier to learn a reliable decision boundary.


```
for s in stage_order],
)))
```

Reason: Stage has a natural order (Pre-seed → Seed → Series A → ...). Ordinal encoding (1,2,3...) encodes this ordering explicitly. One-hot encoding would create 8 binary columns and throw away the ordering information, forcing tree models to re-discover it from scratch.

2.3 Engineer New Predictive Features

```
# CELL 9 – Create new features from domain knowledge

# ① Company Age at time of funding (already exists but needs audit)
df['Company_Age'] = (df['Year'] - df['Founded']).clip(lower=0)

# Cap extreme ages (e.g., founded 1963 = 55yr old company in dataset)
df['Company_Age'] = df['Company_Age'].clip(upper=30)

# ② Is the city a Tier-1 startup hub?
tier1 = ['Bangalore', 'Mumbai', 'Delhi', 'Gurgaon',
         'Hyderabad', 'Chennai', 'Pune', 'Noida']
df['Is_Tier1'] = df['City'].isin(tier1).astype(int)

# ③ Founding decade (captures era effects – 2010s startup boom)
df['Founded_decade'] = (df['Founded'] // 10 * 10).astype('Int64')
df['Is_post2015'] = (df['Founded'] >= 2015).astype(int)

# ④ Stage squared (captures non-linear jump in funding at late stages)
df['Stage_ord_sq'] = df['Stage_ord'] ** 2

# ⑤ Age × Stage interaction
# Older companies at later stages tend to raise much more
df['Age_x_Stage'] = df['Company_Age'].fillna(0) * df['Stage_ord'].fillna(0)

# Show summary of new features
new_features = ['Company_Age', 'Is_Tier1', 'Is_post2015',
               'Stage_ord', 'Stage_ord_sq', 'Age_x_Stage']

display(HTML('<h4>New Feature Summary</h4>'))

display(df[new_features].describe().style.format('{:, .2f}'))
```

Reason: Age × Stage is an interaction feature — a 10-year-old company at Series C is in a very different funding position than a 1-year-old at Seed. Without this interaction term, a linear model cannot capture this multiplicative relationship. Stage_ord_sq captures the fact that funding jumps non-linearly (not just linearly) as startups mature. Is_post2015 captures the structural shift in the Indian startup ecosystem after the 2015 Digital India push that dramatically increased funding activity.

2.4 Build the Final Feature Matrix & Target Variables

```
# CELL 10 – Define X and y for both tasks

# ■■ Core numeric & binary features ■■

base_features = [
```


Reason: stratify=y_clf in the classification split ensures every class appears in the same proportion in train, val, and test. Without stratification, random chance could put all 'Series C+' examples in train and none in test, making evaluation meaningless for that class. We do NOT stratify the regression split because the target is continuous — stratification is only for class labels.

2.5 Visualise Feature Correlations with the Target

```
# CELL 11 – Correlation bar chart
numeric_df = df[base_features + ['log_Amount']].dropna()
corr_with_target = (numeric_df.corr()['log_Amount']
                    .drop('log_Amount')
                    .sort_values(key=abs, ascending=False))

display(HTML('<h4>Feature Correlation with log(Amount)</h4>'))
display(corr_with_target.to_frame('Pearson r').style.bar(
    subset=['Pearson r'], align='zero', color=['#DC2626', '#1B7F79']))

fig, ax = plt.subplots(figsize=(10, 5))
colors_bar = ['#1B7F79' if v > 0 else '#DC2626'
              for v in corr_with_target.values]
ax.barh(corr_with_target.index[::-1],
        corr_with_target.values[::-1],
        color=colors_bar[::-1], edgecolor='white')
ax.axvline(0, color='black', linewidth=0.8)
ax.set_title('Feature Correlation with log(Funding Amount)',
            fontsize=12, fontweight='bold')
ax.set_xlabel('Pearson Correlation Coefficient')
plt.tight_layout()
plt.savefig('feature_correlation.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: A bivariate correlation chart is a fast sanity check before modelling. Stage_ord should show the highest correlation (~0.77) confirming it is the strongest single predictor. If any feature shows near-zero correlation with the target AND is not expected to show non-linear patterns, consider dropping it. Green bars = positive correlation (more → higher funding), red = negative.


```

return {'Model': name, 'RMSE': rmse, 'R2': r2, 'MAE': mae}

def clf_metrics(name, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average='macro', zero_division=0)
    display(HTML(f'''
<h4>{name}</h4>

<b>Accuracy:</b> {acc:.4f} |
<b>Macro F1:</b> {f1:.4f}

'''))
    return {'Model': name, 'Accuracy': acc, 'Macro_F1': f1}

reg_results, clf_results = [], [] # collect all scores here

```

Reason: We define metric helper functions once so every model is evaluated identically. RMSE penalises large errors more than MAE — important because a very wrong prediction about a startup's funding is more damaging than many small errors. R^2 shows how much variance our model explains. Macro F1 averages F1 across all classes equally — critical here because our classes are imbalanced (Seed >> Series C+), and accuracy alone would hide poor performance on minority classes.

3.2 Task A — Regression: Predict log(Funding Amount)

■ Goal: Given a startup's sector, city, stage, age, and year — predict how much funding it will raise (in log-million USD). This directly answers: 'How much can I expect to raise at my stage?'

Model A1 · Ridge Regression — Baseline

```

# CELL 13 — Ridge Regression (regularised linear baseline)
# Pipeline: StandardScaler → Ridge
# StandardScaler is REQUIRED for Ridge because it is sensitive
# to feature magnitudes — Stage_ord (1-8) vs Age_x_Stage (0-240)
# without scaling, Ridge will unfairly penalise large-scale features.

pipe_ridge = Pipeline([
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0, random_state=42)),
])

t0 = time.time()
pipe_ridge.fit(X_tr, y_tr)
ridge_time = round(time.time() - t0, 2)

y_pred_ridge_val = pipe_ridge.predict(X_val)

display(HTML('<h3>Model A1 — Ridge Regression</h3>'))
r = reg_metrics('Ridge Regression (Val)', y_val, y_pred_ridge_val)
r['Train_time'] = ridge_time
reg_results.append(r)

```

```
display(HTML(f'Training time: <b>{ridge_time}s</b>'))

# 5-Fold Cross-Validation
cv_rmse = cross_val_score(
    pipe_ridge, X_tr, y_tr,
    scoring='neg_root_mean_squared_error', cv=5)

display(HTML(f'5-Fold CV RMSE: <b>{-cv_rmse.mean():.4f} ± {cv_rmse.std():.4f}</b>'))
```

Reason: We always start with a simple baseline model. If XGBoost only beats Ridge by 0.02 RMSE but takes 100× longer to train, Ridge is the better production choice. $\alpha=1.0$ is Ridge's L2 regularisation strength — it prevents overfitting by penalising large coefficients. We tune this in Part 4.

Model A2 · Random Forest Regressor

```
# CELL 14 – Random Forest Regressor

# No StandardScaler needed – tree splits are scale-invariant

rf_reg = RandomForestRegressor(
    n_estimators = 200, # number of trees
    max_depth = 12, # max tree depth (prevents overfitting)
    min_samples_leaf = 5, # each leaf needs ≥5 samples
    random_state = 42,
    n_jobs = -1, # use all CPU cores
)

t0 = time.time()
rf_reg.fit(X_tr, y_tr)
rf_time = round(time.time() - t0, 2)

y_pred_rf_val = rf_reg.predict(X_val)

display(HTML('<h3>Model A2 – Random Forest Regressor</h3>'))
r = reg_metrics('Random Forest (Val)', y_val, y_pred_rf_val)
r['Train_time'] = rf_time
reg_results.append(r)

# Feature importance (built-in)
feat_imp = pd.Series(rf_reg.feature_importances_,
    index=ALL_FEATURES).sort_values(ascending=False)

display(HTML('<h4>Top 10 Feature Importances (RF)</h4>'))

display(feat_imp.head(10).to_frame('Importance').style
    .bar(color='#1B7F79').format('{:.4f}'))
```

Reason: $n_estimators=200$ gives stable variance estimates — more trees reduce variance at the cost of training time. $max_depth=12$ and $min_samples_leaf=5$ are regularisation parameters for trees — they prevent individual trees from memorising noise. $n_jobs=-1$ parallelises across all available CPU cores. Random Forest's built-in feature importances give a quick 'preview' of which features matter — we do a deeper analysis with SHAP in Part 5.

Model A3 · XGBoost Regressor

```
# CELL 15 — XGBoost Regressor

# XGBoost is a gradient boosting algorithm that builds trees sequentially,
# each correcting the errors of the previous one.

xgb_reg = XGBRegressor(
    n_estimators = 300,
    learning_rate = 0.05, # small = more trees needed, less overfit
    max_depth = 6, # shallower than RF — boosting compensates
    subsample = 0.8, # use 80% of rows per tree (stochastic)
    colsample_bytree = 0.8, # use 80% of features per tree
    reg_alpha = 0.1, # L1 regularisation
    reg_lambda = 1.0, # L2 regularisation
    random_state = 42,
    eval_metric = 'rmse',
    early_stopping_rounds = 20, # stop if no improvement in 20 rounds
    verbosity = 0,
)

t0 = time.time()
xgb_reg.fit(X_tr, y_tr,
    eval_set=[(X_val, y_val)], # monitor val loss
    verbose=False)
xgb_time = round(time.time() - t0, 2)

y_pred_xgb_val = xgb_reg.predict(X_val)

display(HTML('<h3>Model A3 — XGBoost Regressor</h3>'))
display(HTML(f'Best iteration: <b>{xgb_reg.best_iteration}</b>'))
r = reg_metrics('XGBoost (Val)', y_val, y_pred_xgb_val)
r['Train_time'] = xgb_time
reg_results.append(r)
```

Reason: early_stopping_rounds=20 monitors the validation RMSE and stops training if it hasn't improved for 20 consecutive rounds — this is automatic regularisation that prevents overfitting without manually tuning n_estimators. subsample=0.8 and colsample_bytree=0.8 introduce randomness (like Random Forest does), which reduces variance. learning_rate=0.05 is conservative — a smaller learning rate typically gives better generalisation.

3.3 Task B — Classification: Predict Funding Stage

■ Goal: Given a startup's sector, city, age, year, and the funding they expect to raise — predict which stage (Seed/Series A/B/C+...) the deal is likely to be. This answers: 'Am I positioning my raise at the right stage?'

```
# CELL 16 — Encode string target to integers for XGBoost
```

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
yc_tr_enc = le.fit_transform(yc_tr)
yc_val_enc = le.transform(yc_val)
yc_te_enc = le.transform(yc_te)

display(HTML('<h4>Class Label Encoding</h4>'))

display(dict(zip(le.classes_, le.transform(le.classes_)))

```

Reason: XGBoost requires integer class labels for multi-class classification. LabelEncoder transforms 'Seed' → 0, 'Pre-seed' → 1, etc. We fit it ONLY on the training set (le.fit_transform(yc_tr)) and use le.transform() for val/test to prevent data leakage — the encoder must not 'see' test labels before evaluation.

Model B1 · Logistic Regression — Baseline

```

# CELL 17 – Logistic Regression (multi-class baseline)

pipe_lr = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(
        max_iter = 1000,
        multi_class = 'auto',
        class_weight= 'balanced', # handle imbalance
        random_state= 42
    ))
])

pipe_lr.fit(Xc_tr, yc_tr)
y_pred_lr_val = pipe_lr.predict(Xc_val)

display(HTML('<h3>Model B1 – Logistic Regression</h3>'))

r = clf_metrics('Logistic Regression (Val)', yc_val, y_pred_lr_val)
clf_results.append(r)

display(HTML('<h4>Classification Report</h4>'))
display(HTML('<pre>' + classification_report(
    yc_val, y_pred_lr_val, zero_division=0) + '</pre>'))

```

Model B2 · Random Forest Classifier

```

# CELL 18 – Random Forest Classifier

rf_clf = RandomForestClassifier(
    n_estimators = 200,
    max_depth = 10,
    class_weight = 'balanced', # handles class imbalance
    random_state = 42,
    n_jobs = -1,
)

```

```

rf_clf.fit(Xc_tr, yc_tr)
y_pred_rfc_val = rf_clf.predict(Xc_val)

display(HTML('<h3>Model B2 – Random Forest Classifier</h3>'))
r = clf_metrics('Random Forest (Val)', yc_val, y_pred_rfc_val)
clf_results.append(r)

display(HTML('<h4>Classification Report</h4>'))
display(HTML('<pre>' + classification_report(
yc_val, y_pred_rfc_val, zero_division=0) + '</pre>'))

```

Model B3 · XGBoost Classifier

```

# CELL 19 – XGBoost Classifier
xgb_clf = XGBClassifier(
    n_estimators = 300,
    learning_rate = 0.05,
    max_depth = 6,
    subsample = 0.8,
    colsample_bytree = 0.8,
    use_label_encoder = False,
    eval_metric = 'mlogloss',
    early_stopping_rounds = 20,
    random_state = 42,
    verbosity = 0,
)

xgb_clf.fit(Xc_tr, yc_tr_enc,
            eval_set=[(Xc_val, yc_val_enc)],
            verbose=False)

y_pred_xgbc_val = le.inverse_transform(xgb_clf.predict(Xc_val))

display(HTML('<h3>Model B3 – XGBoost Classifier</h3>'))
display(HTML(f'Best iteration: <b>{xgb_clf.best_iteration}</b>'))
r = clf_metrics('XGBoost (Val)', yc_val, y_pred_xgbc_val)
clf_results.append(r)

```

Reason: `le.inverse_transform()` converts the predicted integer labels (0,1,2...) back to human-readable stage names ('Seed','Series A'...) before passing to `clf_metrics`. This means our metric function always works with string labels — consistent and readable throughout.

3.4 Model Comparison Dashboard

```

# CELL 20 – Regression model comparison
reg_df = pd.DataFrame(reg_results).set_index('Model')

display(HTML('<h3>Regression Model Comparison</h3>'))

```

```

display(reg_df.style
.format({'RMSE': '{:.4f}', 'R2': '{:.4f}',
'MAE': '{:.4f}', 'Train_time': '{:.2f}s'})
.highlight_min(subset=['RMSE', 'MAE'], color='#D1FAE5')
.highlight_max(subset=['R2'], color='#D1FAE5')
.highlight_max(subset=['Train_time'], color='#FEE2E2'))

# Visual comparison
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
metrics_plot = ['RMSE', 'R2', 'MAE']
palette = ['#3B82F6', '#1B7F79', '#E8A838']

for ax, metric, color in zip(axes, metrics_plot, palette):
    ax.bar(reg_df.index, reg_df[metric], color=color, edgecolor='white')
    ax.set_title(f'Regression - {metric}', fontsize=11, fontweight='bold')
    ax.set_xticklabels(reg_df.index, rotation=20, ha='right', fontsize=9)
    for i, v in enumerate(reg_df[metric]):
        ax.text(i, v + 0.001, f'{v:.3f}', ha='center', fontsize=8.5)

plt.suptitle('Regression Model Comparison - Validation Set',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('regression_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

# Classification comparison
clf_df = pd.DataFrame(clf_results).set_index('Model')
display(HTML('<h3>Classification Model Comparison</h3>'))
display(clf_df.style
.format({'Accuracy': '{:.4f}', 'Macro_F1': '{:.4f}'})
.highlight_max(color='#D1FAE5'))

```

Reason: highlight_min on RMSE and MAE (green = lowest = best) alongside highlight_max on R² (green = highest = best) makes the winner immediately obvious without reading numbers. highlight_max on Train_time with red reminds you of the speed-accuracy trade-off. Both regression and classification comparisons are displayed so you can decide which task to take forward to Part 4.

PART 4

Predictions with the Best Model

■ This part assumes XGBoost wins both tasks (the typical outcome on tabular data). If Random Forest wins on your run, replace `xgb_reg` / `xgb_clf` with `rf_reg` / `rf_clf` in every cell below.

4.1 Select the Best Model Programmatically

```
# CELL 21 – Select best regression and classification models

# Pick programmatically so the code works regardless of which model wins

# Regression: lowest RMSE on validation set
best_reg_name = reg_df['RMSE'].idxmin()
best_reg_model = {
    'Ridge Regression (Val)' : pipe_ridge,
    'Random Forest (Val)' : rf_reg,
    'XGBoost (Val)' : xgb_reg,
}[best_reg_name]

display(HTML(f'Best Regressor: <b style="color:#1B7F79">{best_reg_name}</b>'))
display(reg_df.loc[best_reg_name].to_frame('Score'))

# Classification: highest Macro F1 on validation set
best_clf_name = clf_df['Macro_F1'].idxmax()
best_clf_model = {
    'Logistic Regression (Val)': pipe_lr,
    'Random Forest (Val)' : rf_clf,
    'XGBoost (Val)' : xgb_clf,
}[best_clf_name]

display(HTML(f'Best Classifier: <b style="color:#1B7F79">{best_clf_name}</b>'))
display(clf_df.loc[best_clf_name].to_frame('Score'))
```

Reason: We use `.idxmin()` / `.idxmax()` to pick the winner automatically rather than hard-coding a model name. This makes your notebook reproducible — if you add a fourth algorithm later, the 'winner selection' cell still works correctly without any changes.

4.2 Hyperparameter Tuning with GridSearchCV

■ We tune only the best model to save time. GridSearchCV exhaustively tries every parameter combination with 5-fold cross-validation.

```
# CELL 22 – GridSearchCV for XGBoost Regressor

from sklearn.model_selection import GridSearchCV

param_grid = {
```

```

'n_estimators' : [200, 400],
'max_depth' : [4, 6, 8],
'learning_rate' : [0.03, 0.05, 0.10],
'subsample' : [0.7, 0.85],
'colsample_bytree': [0.7, 0.85],
}

# NOTE: GridSearchCV can take 10-30 min on Colab with this grid.
# Use RandomizedSearchCV if you want faster results:
# from sklearn.model_selection import RandomizedSearchCV
# search = RandomizedSearchCV(XGBRegressor(...), param_grid,
# n_iter=30, cv=5, ...)

grid_xgb = GridSearchCV(
XGBRegressor(random_state=42, verbosity=0, eval_metric='rmse'),
param_grid,
scoring = 'neg_root_mean_squared_error',
cv = 5,
n_jobs = -1,
verbose = 1,
refit = True, # re-train on full train set with best params
)

grid_xgb.fit(X_tr, y_tr)

display(HTML('<h4>Best Hyperparameters</h4>'))
display(grid_xgb.best_params_)
display(HTML(f'Best CV RMSE: <b>{-grid_xgb.best_score_: .4f}</b>'))

# Best model is already refitted on full train set
best_xgb = grid_xgb.best_estimator_

```

Reason: `refit=True` tells `GridSearchCV` to automatically re-train the best parameter combination on the ENTIRE training set (not just one fold) after finding the best params. This means `best_estimator_` is ready for immediate prediction on the test set — no extra `.fit()` call needed. We use `neg_root_mean_squared_error` because `sklearn`'s convention is to maximise the scoring function, so RMSE must be negated.

4.3 Final Evaluation on the Held-Out Test Set

■ The test set is evaluated ONLY ONCE — after all tuning is complete. Looking at test results during tuning constitutes data leakage.

```

# CELL 23 – Final test set evaluation (regression)
y_pred_test_reg = best_xgb.predict(X_te)

display(HTML('<h3>Final Test Set – Regression Results</h3>'))

final_reg = reg_metrics('XGBoost (Test Set)', y_te, y_pred_test_reg)

```

```
# Convert log predictions back to actual millions for interpretability
actual_M = np.expml(y_te) # reverse log1p
predicted_M = np.expml(y_pred_test_reg)

comparison = pd.DataFrame({
    'Actual_logAmt' : y_te.values[:10],
    'Predicted_logAmt': y_pred_test_reg[:10],
    'Actual_M ($M)' : actual_M.values[:10],
    'Predicted_M ($M)': predicted_M[:10],
})

display(HTML('<h4>First 10 Predictions vs Actuals</h4>'))
display(comparison.style.format('{:.2f}'))
```

Reason: `np.expml()` is the mathematical inverse of `np.log1p()`. We apply it to both actual and predicted log-amounts to show results in dollar millions — a much more intuitive unit for an entrepreneur than `log(1 + Amount)`. The comparison table lets you visually inspect whether predictions are in the right ballpark before trusting any downstream analysis.

4.4 Residual Analysis — Is the Model Behaving Well?

```
# CELL 24 – Residual plots for regression
residuals = y_te - y_pred_test_reg

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Plot 1: Predicted vs Actual
axes[0].scatter(y_te, y_pred_test_reg, alpha=0.4, color='#3B82F6', s=18)
axes[0].plot([y_te.min(), y_te.max()],
             [y_te.min(), y_te.max()], 'r--', linewidth=1.5)
axes[0].set_title('Predicted vs Actual (log scale)', fontweight='bold')
axes[0].set_xlabel('Actual log(Amount_M)')
axes[0].set_ylabel('Predicted log(Amount_M)')

# Plot 2: Residuals vs Predicted
axes[1].scatter(y_pred_test_reg, residuals, alpha=0.4, color='#1B7F79', s=18)
axes[1].axhline(0, color='red', linestyle='--', linewidth=1.5)
axes[1].set_title('Residuals vs Predicted', fontweight='bold')
axes[1].set_xlabel('Predicted log(Amount_M)')
axes[1].set_ylabel('Residual')

# Plot 3: Residual distribution
axes[2].hist(residuals, bins=40, color='#E8A838', edgecolor='white', alpha=0.85)
axes[2].axvline(0, color='red', linestyle='--', linewidth=1.5)
axes[2].set_title('Residual Distribution', fontweight='bold')
axes[2].set_xlabel('Residual')
```

```
axes[2].set_ylabel('Frequency')

plt.suptitle('Regression Residual Analysis', fontsize=13, fontweight='bold')

plt.tight_layout()

plt.savefig('residual_analysis.png', dpi=150, bbox_inches='tight')

plt.show()
```

Reason: Three residual plots together reveal three different model problems: (1) Predicted vs Actual — dots near the red diagonal = good predictions; a curve means systematic non-linearity the model missed. (2) Residuals vs Predicted — random scatter around zero = healthy; a funnel shape means heteroscedasticity (errors grow with prediction value). (3) Residual distribution — should be approximately normal and centred on zero; a skewed histogram means the model is systematically biased.

4.5 Confusion Matrix & Classification Report

```
# CELL 25 – Classification final test evaluation
y_pred_test_clf = le.inverse_transform(best_clf_model.predict(Xc_te))

display(HTML('<h3>Final Test Set – Classification Results</h3>'))

clf_metrics('XGBoost Classifier (Test Set)', yc_te, y_pred_test_clf)

# Full classification report
display(HTML('<h4>Classification Report (per class)</h4>'))

display(HTML('<pre>' + classification_report(
yc_te, y_pred_test_clf, zero_division=0) + '</pre>'))

# Confusion Matrix
labels = sorted(yc_te.unique())

cm = confusion_matrix(yc_te, y_pred_test_clf, labels=labels)

fig, ax = plt.subplots(figsize=(9, 7))

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)

disp.plot(cmap='Blues', ax=ax, colorbar=False)

ax.set_title('Confusion Matrix – Funding Stage Prediction (Test Set)',
fontsize=12, fontweight='bold')

plt.xticks(rotation=35, ha='right')

plt.tight_layout()

plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')

plt.show()
```

Reason: The confusion matrix shows exactly which stages get confused with which. For example, if 'Pre-Series A' rows get predicted as 'Seed', the matrix reveals this pattern. An entrepreneur using this model wants to know: 'When the model is wrong, how wrong is it?' — a model that confuses adjacent stages (Seed→Series A) is much less dangerous than one that confuses Seed with Series C+.

4.6 Make Predictions on a Custom New Startup

■ This is the most business-relevant cell — it lets an entrepreneur input their own startup's details and get a funding prediction.

[illegible]

```
city = 'Bangalore',
stage = 'Series A',
founded_year = 2019,
predict_year = 2024,
)

display(HTML(f'''
<h3>Prediction for Your Startup</h3>
<b>Sector:</b> EdTech | <b>City:</b> Bangalore<br>
<b>Stage:</b> Series A | <b>Founded:</b> 2019<br><br>
<b>Predicted log(Amount):</b> {log_p:.3f}<br>
<b>Predicted Funding:</b> <span style="color:#1B7F79;font-size:18px">
${amt_p:,.2f}M</span>
'''))
```

PART 5

Best Model Interpretation — SHAP

■ SHAP (SHapley Additive exPlanations) is the gold standard for explaining ML model predictions. It is grounded in cooperative game theory and tells you EXACTLY how much each feature contributed to each individual prediction — not just global averages.

5.1 What is SHAP and Why It Matters for Entrepreneurs

SHAP Concept	What It Means in Practice
SHAP value > 0	This feature PUSHED the prediction UP (more funding predicted)
SHAP value < 0	This feature PUSHED the prediction DOWN (less funding predicted)
SHAP value = 0	This feature had no impact on this specific prediction
SHAP value	The magnitude = how strongly this feature influenced the result
Global importance	Average SHAP across all predictions = feature's overall impact
Local explanation	SHAP values for ONE prediction = why did the model predict THIS?

5.2 Compute SHAP Values

```
# CELL 27 – Compute SHAP values for the best XGBoost regressor
import shap

# TreeExplainer is optimised for tree-based models (XGBoost, RF)
# It is 100-1000x faster than the model-agnostic KernelExplainer
explainer = shap.TreeExplainer(best_xgb)

# Compute SHAP values on the TEST set
# Shape: (n_test_samples, n_features)
shap_values = explainer.shap_values(X_te)

display(HTML(f'SHAP values shape: <b>{shap_values.shape}</b>'))
display(HTML(f'Expected value (model base rate): <b>{explainer.expected_value:.4f}</b>'))
display(HTML('''
<i>The expected_value is the model output if we knew nothing about
the startup – it is the mean prediction across the training set.</i>
'''))

# Quick sanity check:
# SHAP values + expected_value should ≈ model output
predicted_check = best_xgb.predict(X_te[:3])
```

```

shap_sum_check = shap_values[:3].sum(axis=1) + explainer.expected_value

display(HTML('<h4>Sanity Check: SHAP sum ≈ Model Prediction</h4>'))

display(pd.DataFrame({
    'Model Pred' : predicted_check,
    'SHAP Sum' : shap_sum_check,
    'Difference' : predicted_check - shap_sum_check,
}))

```

Reason: The sanity check is essential. SHAP values must sum to exactly (model prediction - expected_value) — this is the SHAP additivity property. If the differences are not near-zero, something is wrong with the explainer setup (e.g., wrong model type passed). TreeExplainer is used here because our best model is XGBoost — it computes exact SHAP values using the tree structure instead of sampling approximations.

5.3 Global Feature Importance — SHAP Summary Plot

```

# CELL 28 – SHAP Summary Plot (the most important SHAP visualisation)

fig, ax = plt.subplots(figsize=(10, 8))

shap.summary_plot(
    shap_values,
    X_te,
    feature_names = ALL_FEATURES,
    max_display = 15, # show top 15 features
    show = False, # suppress auto-show, use plt.show() instead
)

plt.title('SHAP Summary Plot – Top 15 Feature Impacts on log(Amount)',
    fontsize=12, fontweight='bold', pad=12)

plt.tight_layout()

plt.savefig('shap_summary.png', dpi=150, bbox_inches='tight')

plt.show()

display(HTML('''
<b>How to read this plot:</b><br>
• Each dot = one startup in the test set<br>
• X-axis = SHAP value (impact on log funding prediction)<br>
• Colour = feature value (red = high, blue = low)<br>
• Features ranked by mean |SHAP| (most important at top)<br>
• Red dots on the RIGHT → high feature value INCREASES funding prediction
'''))

```

Reason: The SHAP summary plot is the single richest visualisation in the entire ML pipeline. It combines three pieces of information: (1) which features matter most globally (y-axis ranking), (2) the direction of impact for each feature (positive/negative x), and (3) whether the effect is monotone — do high values (red) always increase predictions, or is the relationship more complex? A traditional feature importance bar chart shows only (1). SHAP shows all three simultaneously.

5.4 SHAP Bar Plot — Mean Absolute Impact

```
# CELL 29 — SHAP bar plot (global feature importance)

fig, ax = plt.subplots(figsize=(9, 6))

shap.summary_plot(
    shap_values,
    X_te,
    feature_names = ALL_FEATURES,
    plot_type = 'bar', # mean |SHAP| bar chart
    max_display = 15,
    show = False,
)

plt.title('Mean |SHAP| — Global Feature Importance',
    fontsize=12, fontweight='bold')

plt.tight_layout()

plt.savefig('shap_bar.png', dpi=150, bbox_inches='tight')

plt.show()

# Numerical SHAP importance table
shap_imp = pd.DataFrame({
    'Feature' : ALL_FEATURES,
    'Mean |SHAP|': np.abs(shap_values).mean(axis=0)
}).sort_values('Mean |SHAP|', ascending=False)

display(HTML('<h4>SHAP Importance Table</h4>'))

display(shap_imp.head(15).reset_index(drop=True)
    .style.bar(subset=['Mean |SHAP|'], color='#7C3AED')
    .format({'Mean |SHAP|': '{:.4f}'}))
```

5.5 SHAP Dependence Plots — How Each Feature Works

```
# CELL 30 — Dependence plots for the top 3 features

# A dependence plot shows: as Feature X increases, how does its SHAP value change?
# The colour = a second feature chosen by SHAP to highlight interactions.

top3 = shap_imp['Feature'].head(3).tolist()

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

for ax, feat in zip(axes, top3):
    shap.dependence_plot(
        feat,
        shap_values,
        X_te,
        feature_names = ALL_FEATURES,
```

```

ax = ax,

show = False,

)

ax.set_title(f'SHAP Dependence: {feat}',
fontsize=10, fontweight='bold')

plt.suptitle('SHAP Dependence Plots – Top 3 Features',
fontsize=12, fontweight='bold')

plt.tight_layout()

plt.savefig('shap_dependence.png', dpi=150, bbox_inches='tight')

plt.show()

```

Reason: Dependence plots reveal non-linearity and interaction effects that a simple bar chart hides. For example, Stage_ord might have a large SHAP value for stage=6 (Series C+) but near-zero for stage=2 (Seed) — this non-linear jump in importance would be invisible on a bar chart. The second-feature colour gradient reveals interactions: if the dots are strongly coloured, the impact of Feature X depends on the value of Feature Y.

5.6 Waterfall Plot — Explain a Single Prediction

```

# CELL 31 – Waterfall plot for one test startup

# Choose a specific startup to explain (row 0 of test set)

idx = 0 # change this to inspect any startup

# Build SHAP Explanation object

explanation = shap.Explanation(
    values = shap_values[idx],
    base_values = explainer.expected_value,
    data = X_te.iloc[idx].values,
    feature_names= ALL_FEATURES,
)

# Display startup details

display(HTML(f'<h4>Explaining Prediction for Test Startup #{idx}</h4>'))

display(X_te.iloc[[idx]][['Stage_ord', 'Company_Age',
'Is_Tier1', 'Year']].T)

model_pred = best_xgb.predict(X_te.iloc[[idx]])[0]

display(HTML(f'Model prediction: <b>{model_pred:.3f}</b>
(≈ ${np.exp(1*(model_pred):, .2f}M)'))

# Waterfall plot

fig, ax = plt.subplots(figsize=(10, 6))

shap.plots.waterfall(explanation, max_display=12, show=False)

plt.title(f'Waterfall Plot – Why did the model predict this?',
fontsize=11, fontweight='bold')

```

```
plt.tight_layout()

plt.savefig('shap_waterfall.png', dpi=150, bbox_inches='tight')

plt.show()

display(HTML('''
<b>Reading the waterfall:</b><br>


- Bottom bar =  $E[f(X)]$  = model average (baseline)<br>
- Red bars = features that PUSH prediction UP<br>
- Blue bars = features that PUSH prediction DOWN<br>
- Top bar =  $f(x)$  = this startup's predicted log(Amount)


'''))
```

Reason: `shap.initjs()` loads the required JavaScript — without it, force plots render as blank. `matplotlib=False` renders an interactive HTML object that can be hovered and zoomed in Colab. The multi-sample force plot stacks all 50 test predictions and lets you hover over each one to see its feature contributions — invaluable for spotting patterns across many startups at once. Both waterfall and force plot show the same data; waterfall is clearer for presentations, force plot is better for interactive exploration.

PART 6

Save & Deploy Your Trained Model

6.1 Save the Trained Model with joblib

```
# CELL 33 — Save all model artefacts

import joblib

from google.colab import drive

drive.mount('/content/drive')

SAVE_DIR = '/content/drive/MyDrive/IndianStartupML'

import os; os.makedirs(SAVE_DIR, exist_ok=True)

# Save the best regression model
joblib.dump(best_xgb, f'{SAVE_DIR}/xgb_regressor.joblib')

# Save the label encoder (needed for classification predictions)
joblib.dump(le, f'{SAVE_DIR}/label_encoder.joblib')

# Save the best classifier
joblib.dump(best_clf_model, f'{SAVE_DIR}/xgb_classifier.joblib')

# Save feature column names (critical — must match at inference time)
joblib.dump(ALL_FEATURES, f'{SAVE_DIR}/feature_cols.joblib')

# Save the stage order dictionary
joblib.dump(stage_order, f'{SAVE_DIR}/stage_order.joblib')

display(HTML('<b>All artefacts saved to Google Drive:</b>'))

display(os.listdir(SAVE_DIR))
```

Reason: We save ALL_FEATURES (the list of column names) alongside the model because at inference time, the input DataFrame MUST have exactly the same columns in the same order. Without saving this list, you cannot guarantee the prediction function receives the correct feature layout — a silent mismatch causes completely wrong predictions without any error message.

6.2 Load the Model in a New Session & Predict

```
# CELL 34 — Load and use the saved model in any future notebook

import joblib, numpy as np, pandas as pd

from google.colab import drive

drive.mount('/content/drive')

SAVE_DIR = '/content/drive/MyDrive/IndianStartupML'

# Load all artefacts

model_reg = joblib.load(f'{SAVE_DIR}/xgb_regressor.joblib')

model_clf = joblib.load(f'{SAVE_DIR}/xgb_classifier.joblib')

le = joblib.load(f'{SAVE_DIR}/label_encoder.joblib')

ALL_FEATURES = joblib.load(f'{SAVE_DIR}/feature_cols.joblib')
```

```

stage_order = joblib.load(f'{SAVE_DIR}/stage_order.joblib')

display(HTML('<b>Models loaded successfully!</b>'))

display(HTML(f'Features expected: <b>{len(ALL_FEATURES)}</b>'))

# Make a prediction
log_p, amt_p = predict_startup(
    sector='FinTech', city='Mumbai',
    stage='Series B', founded_year=2017)

display(HTML(f'''
<h4>FinTech · Mumbai · Series B · Founded 2017</h4>
Expected Funding: <b>${amt_p:,.2f}M</b>
'''))

```

6.3 Export All Charts & Results

```

# CELL 35 — Download all outputs to Drive and as ZIP
import shutil, zipfile
from google.colab import files

# Copy all charts to Drive
charts = [f for f in os.listdir('.') if f.endswith('.png')]
for c in charts:
    shutil.copy(c, SAVE_DIR)

display(HTML(f'Copied <b>{len(charts)}</b> charts to Drive'))

# Create downloadable ZIP of all outputs
with zipfile.ZipFile('IndianStartup_ML_Results.zip', 'w') as z:
    for f in os.listdir('.'):
        if f.endswith(('png', 'joblib', 'csv')):
            z.write(f)

files.download('IndianStartup_ML_Results.zip')
display(HTML('<b>Download started!</b>'))

```

COMPLETE NOTEBOOK CELL REFERENCE

Cell	Description	Part
1	Install xgboost, shap; all imports	1
2	Data audit — shape, duplicates, nulls, Stage chaos	1
3	Drop 50 duplicate encoded columns	1
4	Recompute log_Amount (fix corrupted zeros)	1

5	Winsorise Amount_M at 99.5th percentile	1
6	Consolidate 70+ Stage labels → 8 clean categories	1
7	Re-encode Sector & City cleanly with one-hot	2
8	Ordinal encode Stage_clean → Stage_ord	2
9	Engineer: Company_Age, Is_Tier1, Is_post2015, interactions	2
10	Build X_reg, y_reg, X_clf, y_clf; train/val/test splits	2
11	Feature correlation bar chart with log(Amount)	2
12	ML imports; reg_metrics / clf_metrics helper functions	3
13	Model A1: Ridge Regression + 5-fold CV	3
14	Model A2: Random Forest Regressor	3
15	Model A3: XGBoost Regressor with early stopping	3
16	Encode string Stage labels for XGBoost classifier	3
17	Model B1: Logistic Regression (multi-class)	3
18	Model B2: Random Forest Classifier	3
19	Model B3: XGBoost Classifier with early stopping	3
20	Model comparison dashboard (regression + classification)	3
21	Select best model programmatically	4
22	GridSearchCV hyperparameter tuning	4
23	Final test set evaluation + Actual vs Predicted table	4
24	Residual analysis: 3-panel diagnostic plot	4
25	Confusion matrix + full classification report	4
26	predict_startup() custom prediction function	4
27	Compute SHAP values with TreeExplainer + sanity check	5
28	SHAP summary plot (dot cloud)	5
29	SHAP bar plot + importance table	5
30	SHAP dependence plots (top 3 features)	5
31	Waterfall plot — explain one prediction	5

32	Force plot — interactive single + multi prediction	5
33	Save all model artefacts to Google Drive	6
34	Load model and predict in a new session	6
35	Export all charts + ZIP download	6

Indian Startup Funding ML Guide · 35 Colab Cells · Ridge · Random Forest · XGBoost · SHAP · display() throughout